

LLM 预训练之 MLP / FFN

Feed-Forward Network / SwiGLU / GeGLU / Activation Sparsity • 深度研究手册

从 position-wise 非线性变换，到 gated feature routing、知识表征、FFN 宽度分配、稀疏激活与 MoE：为什么“Attention 负责取信息，MLP 负责改造信息”只是起点

核心视角：FFN = 每-token 的高维特征计算器 + 参数容量主体 + 潜在条件计算入口

Research cut-off: 2026-08-13 | 01-B-Agent

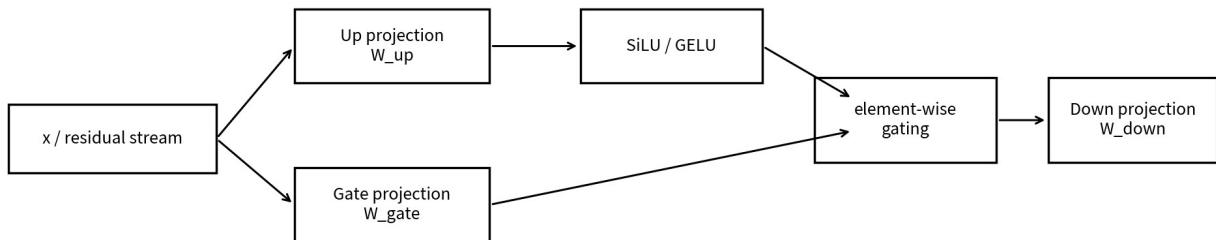
执行摘要：为什么 MLP/FFN 不能被 Attention 的光环遮住

Transformer block 通常由 Attention 与 FFN 两大计算子层构成。Attention 在 token 之间搬运和组合信息，而 FFN 对每个 token 的当前表示独立执行非线性变换。由于 FFN 通常把 d_{model} 扩到更宽的 d_{ff} ，再投影回来，它往往占据 dense Transformer 很大比例的参数与 GEMM 计算。

核心判断	Know-why	工程含义
FFN 是 position-wise，不做 token mixing	同一组权重独立作用于每个位置	它负责“变换/生成特征”，跨 token 通信主要由 Attention 完成
宽度是容量与 FLOPs 的主旋钮	参数量近似随 $d_{\text{model}} * d_{\text{ff}}$ 线性增长	d_{ff} 不应被当成固定 4x 常数
Gated FFN 已成现代主流	GLU variants 在参数/compute 匹配实验中改善 perplexity/下游质量	SwiGLU/GeGLU 需要三矩阵，hidden width 通常相应调整
FFN 与知识/概念关联紧密	多个机制研究在 FFN neuron/vector 上找到模式、事实与 additive concept update	可用于解释/编辑，但不能简化成“一神经元一知识”
FFN 存在显著冗余与稀疏机会	很多 token 只强激活一部分特征	可走 activation sparsity、expert routing、层间共享等路线
每层同宽未必最优	2025-2026 的训练实验开始显示 FFN 重要性随层位置变化	未来更可能是 layerwise capacity allocation，而非 uniform width

最重要的 Know-why Attention 决定当前 token 能从上下文拿到哪些信息；FFN 决定拿到的信息如何被非线性重编码、放大、抑制并写回 residual stream。一个没有 FFN 的 Transformer 仍能路由信息，但缺少了最主要的逐-token 非线性计算容量。

SwiGLU / GeGLU FFN：每个 token 独立执行的“特征生成 + 门控 + 回写”



现代 gated FFN 的本质：先扩展到更宽的 feature space，再用一个内容相关 gate 选择/调制特征，最后投影回 residual stream。

图 1 现代 gated FFN 的数据流：扩展、门控、投影回 residual stream。

1. 原始 Transformer FFN：看起来简单，却是参数大户

原始 Transformer 定义 position-wise feed-forward network 为两次线性变换夹一个 ReLU：

$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$ ，并在序列的每个位置使用相同参数、独立计算。Transformer Base 使用 $d_{\text{model}}=512$ 、 $d_{\text{ff}}=2048$ ，形成经典 4x expansion。 [1]

张量/矩阵	形状	作用
x	[B, L, d]	Attention/Residual 输出的 token state
W_up	[d, m]	从 residual width 扩展到 feature width
activation	[B, L, m]	加入非线性与 feature selectivity
W_down	[m, d]	把 feature update 写回 residual stream

忽略 bias 时，经典两矩阵 FFN 的参数量约为 $2*d*m$ ；如果 $m=4d$ ，则每层 FFN 约为 $8d^2$ 个参数。相比之下，标准 MHA 在 Q/K/V/O 都为 $d \rightarrow d$ 时约为 $4d^2$ 。因此在这种简化形状下，FFN 参数量约是 Attention 投影的两倍。

不要把“position-wise”误解成“简单” FFN 没有 L^2 token-token 交互，但对每个 token 都要运行宽矩阵 GEMM。序列越长、batch 越大，FFN 的总计算也按 token 数线性放大；在大模型预训练中，它可以贡献极大的总 FLOPs。

2. 参数量与 FLOPs：d_ff 才是隐藏的大旋钮

对 classic FFN，矩阵参数约 $2*d*m$ ；对 gated FFN，由于有 up、gate、down 三张矩阵，参数约 $3*d*m$ 。若以“矩阵乘加项”估算 forward compute，二者也分别与 $2*d*m$ 和 $3*d*m$ 成正比。

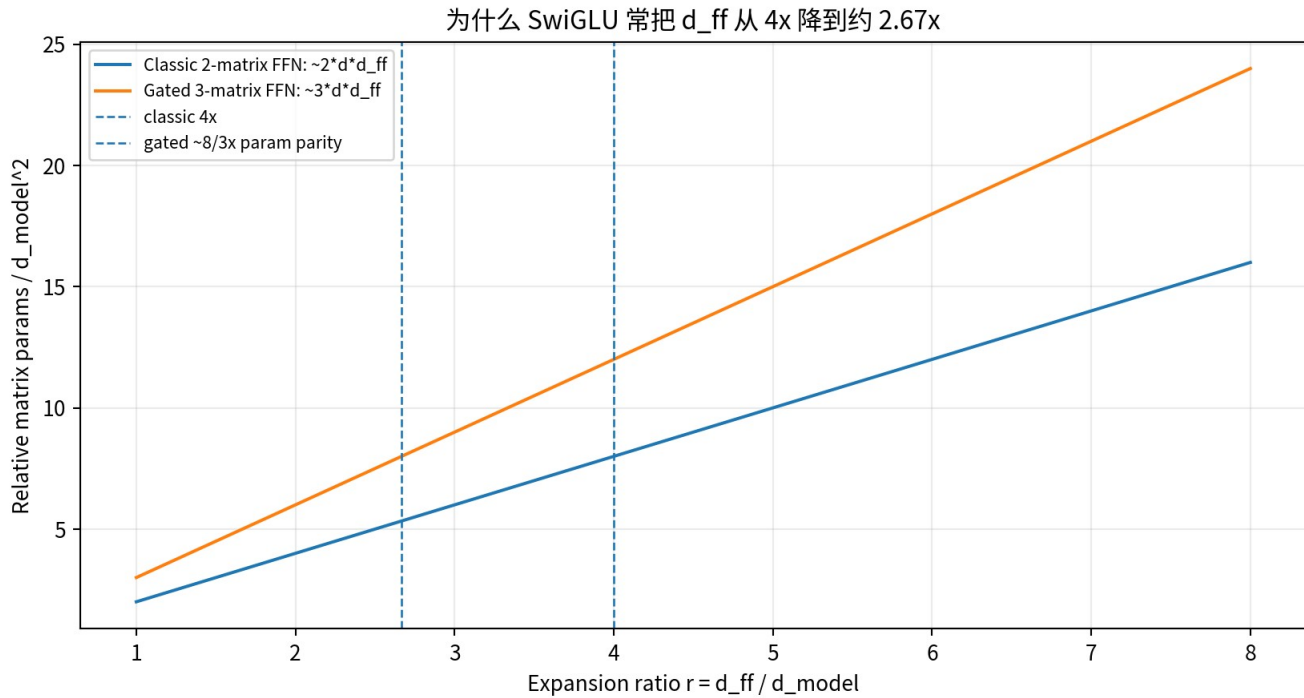


图2 三矩阵 gated FFN 若要与经典 4x 两矩阵 FFN 参数接近， m 常缩到约 $(2/3)*4d = 8d/3$ 。

设计	近似参数	常见 width 直觉	适合比较方式
Classic ReLU/GELU	$2*d*m$	传统 $m \approx 4d$	固定参数/固定 FLOPs 下做 activation ablation
SwiGLU/GeGLU	$3*d*m$	参数匹配时 $m \approx 8d/3$ ；实际模型可偏离	必须同时报告 m 、矩阵数和实际训练 FLOPs
MoE FFN	E 个 expert 参数 + router	总容量大，但每 token 只激活 top-k	区分 total params 与 activated params

3. Activation 演进：ReLU → GELU/SiLU → GLU variants

Activation 的任务不是仅仅“让网络非线性”。它还决定负区间是否保留信息、梯度平滑性、输出尾部、activation sparsity、低精度数值范围以及是否适合硬件跳过零值。

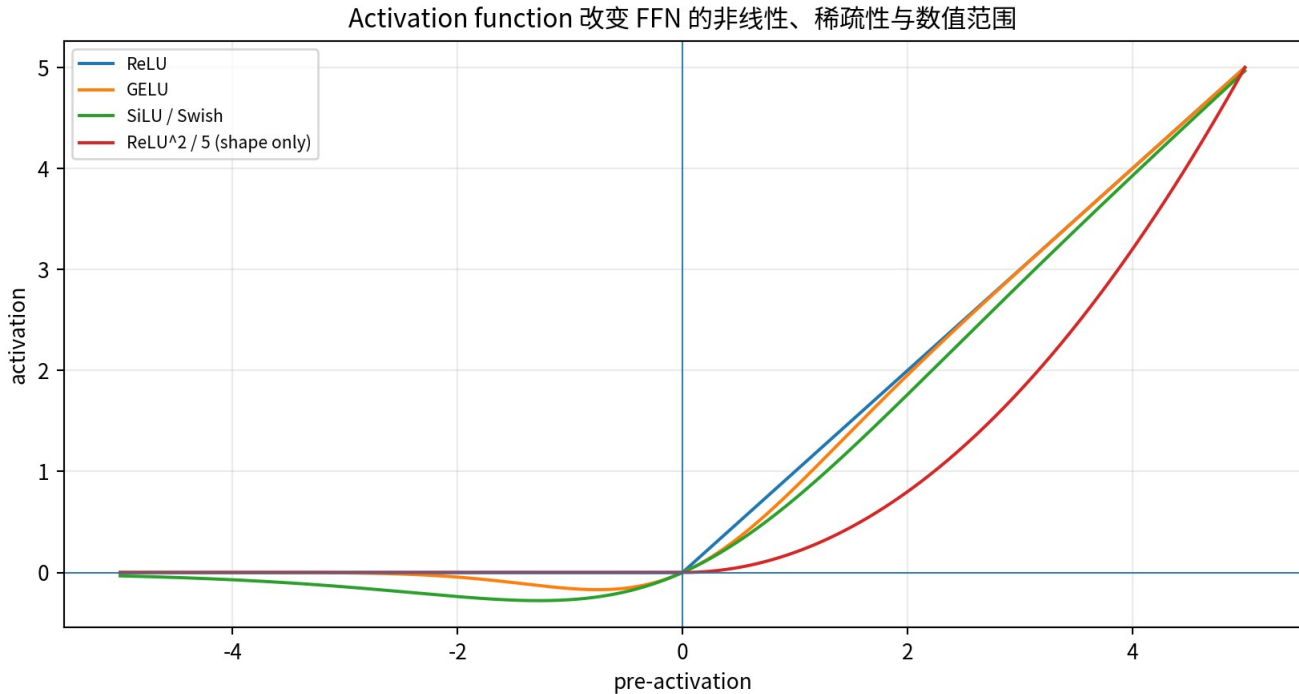


图 3 ReLU、GELU、SiLU 与 ReLU² 的形状示意。Gated FFN 还会把 activation 与另一条线性分支相乘。

Activation	性质	优势	风险/代价
ReLU	负值直接为 0	天然 zero sparsity、简单	不光滑；现代 dense LLM 中常被 gated variants 取代
GELU	平滑概率式门控形状	BERT/GPT 系经典选择	几乎不产生严格零值
SiLU/Swish	$x * \text{sigmoid}(x)$	平滑、自门控感强	正区间可增长较快
ReLU ²	$\max(0, x)^2$	2024 sparse-LLM 研究显示硬件/稀疏潜力	输出增长更快，必须关注稳定性
PowLU	2026 提出的 power-like 稳定激活	目标是降低大规模 SwiGLU spike/outlier	前沿预印本，需要更多独立复现

4. GLU variants：为什么“多一条 gate 分支”会有效

Shazeer 2020 系统比较了 GLU、Bilinear、ReGLU、GEGLU、SwiGLU 等 FFN。核心形式是把一个激活后的投影与另一个线性投影做 element-wise product，再通过 W_{down} 。为公平比较，论文把 gated hidden size 缩到原两矩阵 FFN 的 2/3，使参数与计算近似匹配。[2]

在其 T5-style 预训练实验中，GELU 与 SwiGLU 的 held-out log-perplexity 优于 ReLU/GELU 基线；下游 GLUE/SuperGLUE/SQuAD 也总体更好。这个结果后来影响了 PaLM、LLaMA、Gemma 等多代大模型架构。[2][3][4][5]

形式	简化公式	直觉
Classic	$\phi(xW_1)W_2$	先生成 feature，再统一投影
GELU	$\text{GELU}(xW_g) * (xW_u) \rightarrow W_d$	平滑 gate 调制内容分支
SwiGLU	$\text{SiLU}(xW_g) * (xW_u) \rightarrow W_d$	SiLU gate + 内容分支，现代 decoder LLM 常见

Know-how 比较 activation 时，必须匹配参数量和训练 compute。直接把两矩阵 GELU 的 $m=4d$ 换成三矩阵 SwiGLU 的 $m=4d$ ，会同时增加容量与 FLOPs，实验结论无法归因于 activation 本身。

5. 现代模型的 FFN 设计已经高度分化

LLaMA 明确采用 SwiGLU 取代 ReLU，并按 Shazeer 的思路把 FFN hidden dimension 从传统 $4d$ 调整为约 $2/3 \cdot 4d$ ，再按硬件友好倍数取整。[3]

Gemma 2 的技术报告公开列出 GeGLU 非线性和大幅扩展的 feedforward dimension；同时其 9B ablation 显示，在同等参数预算下，更深而不是更宽的网络在 4 个 benchmark 平均上略优。[4]

PaLM 采用 SwiGLU、无 bias、Pre-LN，并使用 parallel block formulation；其公开配置保持 $d_{\text{ff}}=4 \cdot d_{\text{model}}$ ，说明 gated FFN 的“最优 expansion ratio”并非一个跨模型固定常数。[5]

模型/工作	FFN 特征	可迁移结论
Transformer 2017	ReLU, $4x$	$4x$ 是历史基线，不是理论常数
PaLM 2022	SwiGLU, $d_{\text{ff}}=4d$	大规模模型可选择更宽 gated FFN
LLaMA 2023	SwiGLU, 约 $8/3 d$ 后取整	参数/compute parity 是重要设计原则
Gemma 2 2024	GeGLU, 公开的大 FF dimension	depth/width allocation 必须用 ablation 决定

6. Attention vs FFN：信息路由与特征计算不是绝对分工

维度	Attention	FFN / MLP
跨 token 通信	核心能力：每个位置读取其他位置	默认无跨 token mixing
非线性	softmax/normalization 提供非线性，但主体是加权路由	强非线性 feature transform 是核心
参数规模	投影矩阵通常较少	宽中间层常占更大参数份额
上下文长度成本	dense attention 随 L^2	按 token 数 L 线性增长
知识/模式	可从上下文动态读取	可把训练中形成的模式/事实编码进参数化 feature transformations
稀疏化路线	token/block routing	neuron/expert routing

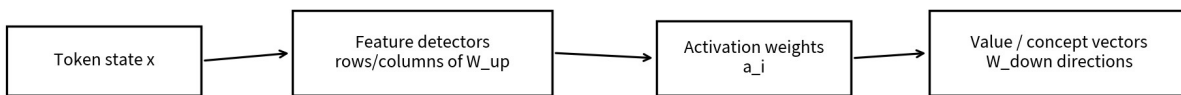
“Attention 负责 retrieval，FFN 负责 memory” 是一个好用但过度简化的二分法。Attention 参数也会学习语义变换；FFN 也会通过 residual 与多层堆叠间接参与跨 token 的推理。更准确的说法是：Attention 提供显式 content-dependent token mixing，FFN 提供共享的 position-wise nonlinear transform。

7. FFN as Key-Value Memory：为什么这个视角很有影响力

Geva et al. 2021 研究指出，Transformer FFN 可解释为 key-value memories：某些“key”方向与训练文本模式相关，激活后对应“value”方向会在输出词表上提升特定后续 token 的概率；浅层更多捕获表面模式，深层更偏语义。[6]

2022 的后续工作把 FFN output 视为对 vocabulary-space distribution 的 additive update，并把单个 parameter vectors 解释为可促进某些人类可读概念的 sub-updates。[7]

FFN as key-value memory：有用的机制视角，但不要字面化



Geva et al. 的解释：某些 hidden units 像 pattern detectors；被激活后，对 residual/vocabulary space 施加特定方向的 additive update。

重要 caveat：这是解释性模型，不等于“一个 neuron = 一条知识”。事实/技能通常仍跨神经元、跨层、跨 Attention/FFN 分布。

图 4 FFN 作为 key-value memory 的机制化解释，以及必须保留的 caveat。

非常重要的 Caveat 这些工作说明 FFN 与知识/模式存在强因果和可解释关联，但不能推出“所有知识只存在 FFN”或“一条事实 = 一个 neuron”。现代模型中的知识往往跨多个层、多个向量，并与 Attention、embedding、residual dynamics 共同实现。

8. Knowledge Neurons 与 ROME：FFN 为什么成为模型编辑入口

Knowledge Neurons 在 BERT 上用 attribution 方法定位与特定事实表达强相关的 FFN neurons，并展示抑制/增强这些 neurons 可以改变事实输出。该工作本身将结论称为 preliminary studies，证据边界应保留。[8]

ROME 通过 causal tracing 在 autoregressive GPT 中找到中层 feed-forward modules 对事实关联的重要作用，并通过 rank-one update 修改特定事实。它支持“中层 MLP 对 factual association recall 很关键”，但同样不能解释成知识只存储在某一个矩阵位置。[9]

证据类型	支持什么	不支持什么
activation attribution	某些 neuron activation 与事实表达相关	该 neuron 独占该事实
causal tracing	特定层/模块对预测有因果贡献	所有任务都依赖同一层位

证据类型	支持什么	不支持什么
weight editing	局部修改可改变事实输出	模型参数是简单键值数据库
vocabulary projection	FFN update 可对应可读概念方向	每个方向都有唯一、固定语义

9. Activation Sparsity：FFN 的“惰性神经元”现象

MoEfication 观察到预训练 Transformer FFN 对很多输入只强激活少量 neurons，并尝试把 dense FFN 自动拆成多个 functional partitions，再由 router 选择部分 expert；其论文在多种模型上以 10%-30% FFN 参数保持超过 95% 原性能。[10]

2024 的 ReLU^2 Wins 与 2025 的 Spark Transformer 延续这条路线：目标是让 FFN activation sparsity 既可预测、又能落到硬件跳过计算上。Spark 在其预训练设置中只激活约 8% FFN neurons，同时维持有竞争力的质量。[16][17]

从 Dense FFN 到 Activation Sparsity 与 MoE：容量和每-token compute 解耦



三者都在回答同一问题：FFN 容量很大，但一个 token 是否真的需要访问全部容量？

图 5 Dense、activation-sparse 与 MoE FFN 都在尝试把总容量与每-token compute 解耦。

稀疏方式	何时决定	节省什么	主要难点
Natural activation sparsity	activation 后	潜在 GEMM/访存	硬件未必能高效利用不规则零值
Top-k neuron sparsity	显式选择 hidden channels	activated neurons	排序/预测器成本，训练质量
MoE routing	进入 expert 前	大部分 expert compute	all-to-all、负载均衡、capacity overflow
Structured 2:4 / n:m	训练/权重布局	Tensor Core 稀疏 GEMM	结构约束可能损害表达

10. Dense FFN → MoE：为什么 MoE 本质上是“把 FFN 复制后路由”

Switch Transformer 把每个 token 只路由到单个 FFN expert，以稀疏激活扩展总参数而维持每-token compute 近似不随 expert 总数线性增长。其核心创新不是 Attention，而是条件化 FFN 容量。[11]

因此 MoE 最自然的理解不是“另一种 Transformer”，而是：把 dense block 中最占参数的 FFN 复制成一组 experts，再加一个 router。Attention 仍可保持 dense 或用 GQA/MLA；FFN 则变成条件计算。

Dense FFN	MoE FFN
所有 token 走同一 $W_{up}/W_{gate}/W_{down}$	token 根据 router 只走 top-k experts
参数量 $\approx$ activated params	total params $\gg$ activated params
通信模式主要是 tensor/data parallel	增加 token dispatch / all-to-all
训练简单稳定	要解决 load balance、expert collapse、capacity factor

11. FFN 宽度：为什么“4x / 8/3x”不该被当成定律

FFN expansion ratio 本质上在分配模型的“per-token nonlinear latent space”。增宽会增加 feature basis、参数和 FLOPs；但如果新增维度只是低能量或冗余方向，边际收益会递减。

EMNLP 2025 的 spectral scaling 研究报告：FFN soft rank 随 width 呈近似 power-law 增长，但 hard rank 增长更慢，说明继续增宽可能主要增加低能量 tail directions，而 dominant subspace 更早饱和。[14]

Apple 2023 的 One Wide Feedforward 还发现，在其 encoder-decoder setting 中，多层 FFN 存在显著冗余；共享一个更宽 FFN 可以在特定设置下获得更好的准确率/延迟 trade-off。[15]

Know-how 不要只 sweep d_{ff}/d_{model} 。更有价值的架构搜索是联合 sweep：model width d 、layer depth N 、FFN ratio r 、activation/gating、attention width、训练 tokens 与总 FLOPs。否则容易把“参数重新分配”误判成“某个 FFN ratio 天生更好”。

12. Layerwise FFN Allocation：每层同宽正在被重新审视

COLM 2025 的训练-from-scratch 实验在 285M、570M、1.2B 模型上重新分配 FFN 容量，报告把 FFN 集中在连续约 70% 的中间层，比统一配置在多项下游任务上更好。结论提示 FFN 的边际价值可能随深度变化。[13]

2026 年 8 月的 Geometry-Guided Layerwise FFN Width Allocation 更进一步：使用 forward-pass 几何变化作为 layerwise capacity proxy，并在 128M/256M/440M 配对训练里报告部分非均匀 width schedule 优于 uniform。该工作非常新，应视为前沿信号而非已定型结论。[20]

Uniform width 的隐含假设	新研究挑战
每层对 FFN 容量需求相同	不同深度承担的表征变换不同
d_{ff} 只需按模型规模统一放大	可以在固定总参数下重新分配到更敏感层
增加 width 的边际收益一致	spectral utilization 与 perturbation sensitivity 会随层变化
结构搜索只需全局 ratio	未来可能需要 layerwise schedule / architecture search

13. FFN Shape：窄-宽-窄也不是不可改

2026 的 Revisiting the Shape Convention 直接挑战传统 narrow-wide-narrow FFN，提出更深的 hourglass sub-MLP，并在 113M-1B 的匹配预算实验中发现：到 400M 时可优于传统 FFN，较大规模到 1B 仍有竞争力。它同时发现 width-depth 存在 U-shaped trade-off。[19]

这类结果的意义不在于“hourglass 已取代 SwiGLU”，而是说明 FFN 结构设计仍远未收敛：我们长期把 2-layer narrow-wide-narrow 当作默认，更像历史路径依赖，而不是由理论证明的唯一最优结构。

14. 训练稳定性：SwiGLU 的表达力也会放大 outlier

Gated FFN 存在乘法路径：gate branch 与 up branch 相乘。若两个分支都出现大幅值，输出可被显著放大，因此训练稳定性需要关注 pre-activation、gate、product、down-projection output 的 RMS/max。

2026 的 PowLU 工作指出，大正输入下 SwiGLU 可能呈近似二次放大，低精度大规模预训练时会加剧 spikes/outliers，并提出更温和的 power-like activation。其 7.9B/124B 实验报告与 SwiGLU 竞争的质量和更好的稳定性，但这是很新的预印本，尚需更多复现。[18]

同年一项硬件导向工作尝试在 FFN 中组合 2:4 weight sparsity 与 activation sparsity，并用“早期稀疏训练 + 末期 dense recovery”达到相近质量，在其硬件/模型设置下报告 1.4-1.7x end-to-end pretraining speedup。[21]

监控项	为什么看	异常信号
up/gate pre-activation RMS max	发现输入 scale 漂移	层间突然增长、长尾极端值
gate output distribution	看门控是否失去选择性	几乎全开或全关
product RMS max	乘法路径的核心风险点	spike 与 NaN 前兆
down projection output norm	看 residual update 规模	远大于 residual stream
per-layer grad norm	定位 unstable FFN layer	少数层长期主导 gradient

15. Tensor Parallel：为什么 FFN 特别适合切矩阵

Megatron-LM 的经典 tensor parallel 做法正好贴合 FFN：第一张大线性层按输出维度做 column parallel，各 GPU 得到一部分 hidden features；activation 在本地完成；第二张线性层按输入维度做 row parallel，再做一次通信归并。这样避免在 FFN 中间先 all-gather 完整宽激活。[12]

当前 Megatron Core 仍把 ColumnParallelLinear 和 RowParallelLinear 作为基础 primitive，并支持把 bias/activation、RMSNorm/linear、reduce-scatter 等做进一步 fusion。[22]

阶段	Tensor Parallel 直觉	主要通信
W_up / W_gate	按 d_ff 维切分，每张卡算一部分 neurons	通常无需立刻 gather 完整 hidden
activation/gating	每 rank 本地 element-wise	无或很少
W_down	每 rank 消化自己的 hidden shard	对输出做 all-reduce / reduce-scatter
sequence parallel 配合	把 LayerNorm/activation 按 sequence 分片	进一步降低 activation memory

系统视角 FFN 大多是规则 GEMM，因此理论 FLOPs 很高但也更容易接近硬件峰值；Attention 的难点更多是 L^2 、softmax、mask 与 IO。大模型训练效率常取决于两者能否在同一并行拓扑中都保持高利用率。

16. FFN Kernel / Fusion：三矩阵意味着更多内存流量机会

SwiGLU/GeGLU 需要 up/gate 两个 projection，再做 element-wise gating，最后 down projection。高效实现通常会尽量融合 gate activation、乘法、bias（若有）、dropout（若有），并避免中间张量频繁写回 HBM。

优化	减少什么	注意
fused SwiGLU kernel	kernel launch + 中间 HBM traffic	数值近似与 dtype 必须验证
Tensor Core friendly d_ff	padding/低效 tile	hidden dim 常按 128/256 等倍数取整
activation checkpoint	保存的 FFN intermediate	增加 backward recompute FLOPs
FP8 / low precision GEMM	memory + compute	gate/product outlier 对 scaling 更敏感
grouped GEMM for MoE	很多小 expert GEMM 启动开销	需要足够 token/expert 批量才能高效

17. 严格 FFN Ablation：怎样避免“换 activation 就赢了”的假实验

必须控制	原因
总参数 / non-embedding params	FFN width 改动会直接改变容量
训练 FLOPs / tokens	更大 FFN 每 token 更贵，固定 step 不公平
d_model / layers / attention config	防止参数从 Attention 转给 FFN 造成混淆
optimizer / LR / warmup	不同 activation 的稳定范围可能不同
dtype / kernel maturity	wall-clock 可能是实现差异而非架构差异
sequence length / data mixture	FFN 与 Attention 的相对收益会随任务分布变化
多 seed + loss curve	小模型架构差异可能与方差同量级

建议同时报告：validation loss vs tokens、loss vs FLOPs、wall-clock、tokens/s、MFU、peak memory、per-layer activation/gradient stats、downstream capability matrix。只有这样才能区分“质量更好”“算力更省”“硬件更快”三种不同胜利。

18. Production FFN Blueprint：从架构选择到可观测性

阶段	推荐动作	产物
1. Baseline	固定 d_model/layers/token budget，建立 GELU 或 SwiGLU baseline	可复现实验基线
2. Activation sweep	ReLU/GELU/SwiGLU/GeGLU/候选激活，做参数/FLOPs 匹配	activation frontier
3. Width sweep	联合搜索 r=d_ff/d、深度和 attention width	compute-optimal shape
4. Stability	监控 gate/product/outlier/grad norm	稳定区间与 clipping/scale recipe
5. Systems	Tensor Parallel + fused gated kernel + recompute	真实 tokens/s/MFU
6. Sparsity	评估 neuron sparsity、top-k、MoE 的收益	activated-parameter frontier
7. Layerwise allocation	固定总 FFN 参数重新分配到层	capacity schedule
8. Long run	用真实 token horizon 验证 small-scale proxy	最终架构决策

推荐默认起点 对于新 dense decoder-only LLM：Pre-RMSNorm + bias-free gated FFN（常见为 SwiGLU）是成熟起点；但 d_ff ratio 应通过参数/FLOPs 匹配 sweep 决定，而不是机械复制某个模型。规模扩大后再验证 activation outlier、layerwise width 与稀疏化机会。

19. 方法选择决策表

目标	优先研究	为什么
通用 dense 质量	SwiGLU/GeGLU + width/depth sweep	成熟、强 baseline
训练稳定性/低精度	activation range、norm、PowLU/SwiGLU clip 等	乘法 gating 容易放大 outlier
降低推理 FLOPs	activation sparsity / top-k neurons	只计算少量 hidden channels
扩大总容量	MoE experts	total params 与 activated params 解耦
减少参数存储	shared FFN / factorized / narrower FFN	利用 FFN 冗余
重新分配固定预算	layerwise width / deeper FFN / hourglass	uniform narrow-wide-narrow 未必最优
大规模训练吞吐	TP column->row + fused gated kernels	FFN 是规则大 GEMM，系统优化收益大

20. Know-Why / Know-How 总结

Know-why	Know-how
Attention 解决 token mixing，FFN 解决 position-wise nonlinear compute	把跨 token 路由和逐 token 特征变换分开诊断
FFN 宽度直接决定参数和 GEMM FLOPs	任何 activation/width 比较都做参数与 FLOPs matching
gating 改善表达但增加一张矩阵和乘法路径	监控 gate/product scale，并用 8/3x 作为参数匹配参考而非定律
FFN 与事实/概念有强机制关联	用 attribution/intervention/editing，避免“一神经元一知识”叙事
FFN 有明显冗余和稀疏性	把 sparse neurons、MoE、shared FFN 视为同一容量效率问题的不同解法
uniform FFN width 是历史默认，不是理论最优	开展 layerwise allocation 和 shape search
FFN 是 tensor parallel 最友好的模块之一	column-parallel up/gate + row-parallel down，融合 activation/communication

最终心智模型 Attention 像“把外部上下文送到当前 token 的通信网络”；FFN 像“每个 token 都共享的一座高维计算工厂”。Up/Gate 负责产生与选择 feature，Down 负责把 feature 结果写回 residual stream；MoE/稀疏 FFN 则是在决定每个 token 只进入工厂中的哪几条生产线。

参考资料与证据等级

- [1] Vaswani et al. (2017), Attention Is All You Need. [\[奠基论文 / 一手\] source](#)
- [2] Shazeer (2020), GLU Variants Improve Transformer. [\[Gated FFN / 一手\] source](#)
- [3] Touvron et al. (2023), LLaMA: Open and Efficient Foundation Language Models. [\[模型技术报告 / 一手\] source](#)
- [4] Gemma Team (2024), Gemma 2: Improving Open Language Models at a Practical Size. [\[模型技术报告 / 一手\] source](#)
- [5] Chowdhery et al. (2022), PaLM: Scaling Language Modeling with Pathways. [\[模型技术报告 / 一手\] source](#)
- [6] Geva et al. (2021), Transformer Feed-Forward Layers Are Key-Value Memories. [\[机制研究 / 同行评审\] source](#)
- [7] Geva et al. (2022), Transformer Feed-Forward Layers Build Predictions by Promoting Concepts in the Vocabulary Space. [\[机制研究 / 同行评审\] source](#)

- [8] Dai et al. (2022), Knowledge Neurons in Pretrained Transformers. [\[机制研究 / 同行评审\] source](#)
- [9] Meng et al. (2022), Locating and Editing Factual Associations in GPT. [\[ROME / 一手\] source](#)
- [10] Zhang et al. (2022), MoEification: Transformer Feed-forward Layers are Mixtures of Experts. [\[FFN 稀疏 / 同行评审\] source](#)
- [11] Fedus et al. (2021), Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. [\[MoE / 一手\] source](#)
- [12] Shoeybi et al. (2019), Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. [\[Tensor Parallel / 一手\] source](#)
- [13] Ikeda et al. (2025), Layerwise Importance Analysis of Feed-Forward Networks in Transformer-based Language Models. [\[COLM 2025 / 同行评审\] source](#)
- [14] Jha & Reagen (2025), Spectral Scaling Laws in Language Models: How Effectively Do Feed-Forward Networks Use Their Latent Space? [\[EMNLP 2025 / 同行评审\] source](#)
- [15] Pires et al. (2023), One Wide Feedforward is All You Need. [\[FFN sharing / 同行评审\] source](#)
- [16] Zhang et al. (2024), ReLU^2 Wins: Discovering Efficient Activation Functions for Sparse LLMs. [\[Sparse activation / 一手\] source](#)
- [17] You et al. (2025), Spark Transformer: Reactivating Sparsity in FFN and Attention. [\[Sparse architecture / 预印本\] source](#)
- [18] Jiang et al. (2026), PowLU: An Activation Function for Stable Pre-Training of LLMs. [\[前沿 activation / 预印本\] source](#)
- [19] Liao et al. (2026), Revisiting the Shape Convention of Transformer Language Models. [\[FFN shape / 预印本\] source](#)
- [20] Mudarisov et al. (2026), Geometry-Guided Layerwise FFN Width Allocation in Transformers. [\[最新前沿 / 预印本\] source](#)
- [21] Madhyastha et al. (2026), To 2:4 Sparsity and Beyond: Neuron-level Activation Function to Accelerate LLM Pre-Training. [\[训练稀疏 / 预印本\] source](#)
- [22] NVIDIA Megatron Core, Tensor Parallel Layers documentation. [\[官方实现文档 / 一手\] source](#)

说明：本手册中的机制解释与工程建议，是基于上述论文和官方技术资料的综合分析；任何 perplexity、benchmark、稀疏率、加速比或稳定性数字都只属于对应论文模型、数据、硬件和训练设置。2026 年的若干结果仍是新近预印本，已在证据等级中单独标注。